

Ontology-Constrained Vector Retrieval

Joseph Natter

Department of Computer Science
Kennesaw State University
Marietta, GA, USA
jnatter@students.kennesaw.edu

Nicholas Hodge

Department of Computer Science
Kennesaw State University
Marietta, GA, USA
nhodge8@students.kennesaw.edu

Abstract—This paper presents an ontology-constrained vector retrieval framework that integrates symbolic ontological structure with high-dimensional vector representations. While vector-based retrieval methods excel at capturing semantic similarity, they lack mechanisms for enforcing structural and logical constraints, often returning semantically related but invalid results.

We implement a pre-hoc constraint approach using ontology-derived type filtering over a vector database (Qdrant) and evaluate it on NASA’s Air Traffic Management ontology, consisting of 2.9 million RDF triples and over 36,000 indexed entities. Compared to unconstrained retrieval, our method improves type validity (valid@5) from 0.82 to 1.00 across a set of realistic queries, with only modest latency overhead.

These results demonstrate that integrating ontology constraints directly into the retrieval process provides a practical and effective way to enforce semantic correctness in vector search systems.

Index Terms—ontology, vectors, databases

I. INTRODUCTION

Database management systems have long served as the foundational infrastructure for data-intensive applications, providing efficient mechanisms for storing, indexing, and querying large collections of structured data. However, the increasing reliance of modern machine learning pipelines on high-dimensional embedding representations has exposed fundamental limitations of traditional relational and key-value data models, which are not designed to support similarity-based retrieval in continuous vector spaces.

Vector databases have emerged to address this gap by offering scalable approximate nearest-neighbor search and vector-native indexing structures, enabling efficient retrieval over large embedding collections. As a result, vector database systems have seen rapid adoption across a range of machine learning workloads, including semantic search, recommendation systems, and representation-based retrieval pipelines, and are now widely recognized as a distinct and important class of data management systems [1], [2]. These systems now underpin a wide range of workloads beyond natural language processing, including recommendation systems, computer vision, and scientific and sensor data management. By organizing data primarily through geometric similarity in embedding space, vector databases offer scalable approximate nearest-neighbor search, but provide limited guarantees about the semantic correctness of retrieved results.

In many real-world applications, however, retrieved results must satisfy domain-specific semantic constraints, such as

type compatibility, hierarchical relationships, or disjointness conditions, which are naturally expressed through ontologies. Existing vector databases typically enforce such constraints, if at all, through post-hoc metadata filtering or application-level logic, which is both inefficient and insufficient to guarantee semantic validity under approximate retrieval. This paper argues that semantic constraints should be foregrounded in the process of vector retrieval and presents an ontology-constrained vector retrieval framework that integrates ontological reasoning directly into the retrieval process, rather than as an external filtering step.

Our contributions are: (i) a formalization of ontology-constrained vector retrieval, (ii) a practical implementation using pre-hoc filtering in a vector database, and (iii) an empirical evaluation demonstrating improved semantic validity with minimal overhead.

II. RELATED WORK

A. Ontology

The term ontology comes from philosophy, where it is the “science of what is, of the kinds and structures of objects, properties, events, processes, and relations in every area of reality” [3].

A natural question is why such representations are useful.

The first answer to this motivation is that automated natural language is naturally “limited by the ambiguity of language” [4]. Any formal framework for conceptualization, especially with the wide scope of “everything” is a crucial and ambitious step for the field of knowledge representation.

The second answer to why we would want to represent data using ontologies is that it is the natural next step for “research in computer science, artificial intelligence, philosophy, library science, and linguistics,” as these fields have already been working to create “standard descriptions and terminology for the entities and events that make up our world” [4].

In other words, ontological structuring in a formal will be the ideal formatting for general knowledge representation as it is not limited by natural language, and is a natural fit for many fields concerned with knowledge representation.

B. Ontology Databases

Modern ontology databases are typically built upon the Resource Description Framework (RDF), a graph-based data model in which all information is represented as triples of the

form $(subject, predicate, object)$. Collectively, RDF triples form a labeled directed graph and are queried using SPARQL, a declarative pattern-matching language whose execution resembles relational join processing over graph-structured data.

Efficient storage and evaluation of large RDF repositories has been extensively studied. Systems such as RDF-3X [5] demonstrate that schema-free graph data can be managed at scale using relational-style indexing and cost-based optimization. RDF-3X employs a single triples table architecture together with exhaustive indexing over all six permutations of subject, predicate, and object, enabling searches by any of the RDF attributes, pairs, or triplets. This work shows that RDF data can be stored compactly and queried efficiently without requiring workload-specific physical design tuning.

While RDF defines a flexible representation for graph data, it does not by itself impose rich semantic constraints. Formal semantics are introduced through RDF Schema (RDFS) and the Web Ontology Language (OWL) [6], [7]. OWL is grounded in description logics (DL) and enables the specification of class hierarchies, property characteristics, disjointness axioms, and domain and range restrictions under formally defined semantics. In ontology-based systems, the ontology defines a conceptual schema layer (TBox), while instance data populates this structure as assertions (ABox).

Ontology databases therefore combine: (i) a triple-based physical storage layer (RDF), (ii) a formally defined logical vocabulary and reasoning layer (OWL), and (iii) a query interface (SPARQL) capable of retrieving both asserted and entailed facts.

However, despite their expressive semantic guarantees, RDF/OWL systems are optimized for symbolic reasoning and graph-pattern matching rather than similarity-based retrieval in high-dimensional embedding spaces. They provide logical validity under formal semantics but do not natively support geometric similarity search or approximate nearest-neighbor retrieval. This limitation motivates the integration of ontological reasoning with vector-native database systems.

C. Vector Database

Vector databases (VDBs) are a popular solution to the problem of processing and handling copious amounts of data in different modalities. VDBs encode various forms of data into vectors and store them in high dimensional formats; the payoff is that vectors are able to represent not only the data itself but the semantic and contextual information behind it. VDBs are behind many of the applications in both specifically large language models (LLMs) and greater artificial intelligence (AI) overall, due to their efficiency, storage capability, and scalability [8].

D. Converting an OWL Document to a Relational Database

There has been a direct attempt to design an algorithm that can convert a given OWL document to a relational database, dubbed *OWL2DB*, designed by Gali, et. al in [9]. Motivated by the success behind SQL and relational database systems, this work was done in direct contrast to a popular alternative

approach, which is to convert OWL to extensible markup language (XML).

There were two major parts of their work: the OWL2DB algorithm and the mapping system.

The mapping system is responsible for the mapping between OWL and the relational schema, and is broken into three parts: Ontology Model, Document Manager, and Ontology Resource Manager.

The ontology model takes OWL documents and outputs a comprehensive ontology model based on the type of OWL document provided: OWL DL, OWL full, or OWL Lite. The document manager can process multiple OWL documents, and tracks where statements from the OWL came from. The ontology reasoner is the final part of the process, responsible for using the resource description framework [10] to each resource from the ontology.

The second part of the pipeline is the OWL2DB algorithm. The algorithm accepts an OWL ontology and converts it into a relational database schema by first parsing the document to identify root classes and traversing their subclass hierarchies to determine depth. As the class graph is explored, subclass depth is tracked, and when a depth of three is reached, relation tables are generated for higher-level subclasses, using class attributes and instances as column names. This process is repeated throughout the hierarchy to collect table names and column definitions, while ensuring that union, complement, and disjoint subclasses at the same level are handled consistently, with disjoint classes mapped to separate relations. The ontology is then parsed again to extract instance values, after which the database tables are created and populated.

This work is relevant for two primary reasons: 1) our dataset is built using OWL full documents, which can be converted using the OWL2DB algorithm and 2) the *ontology modeler* portion of the mapping system could be used in conjunction with vector embeddings, accomplishing our goal of placing the ontology into a pre-hoc or ad-hoc solution.

Ultimately, the “conclusions” derived by the authors lacked comprehensive benchmarking. We extend this line of work by evaluating retrieval performance using quantitative metrics.

III. PROBLEM SETTING

We consider the problem of semantic retrieval over a collection of entities derived from an ontology. Let $\mathcal{E}$ denote a set of entities extracted from an ontology-backed dataset, where each entity $e \in \mathcal{E}$ is associated with: (i) a textual representation t_e , and (ii) a set of ontology types $\mathcal{T}_e$ defined by class assertions and their transitive closure under `rdfs:subClassOf`.

Given a natural language query q , standard vector retrieval embeds both q and each t_e into a shared embedding space and returns the top- k entities according to a similarity function:

$$\text{Top}_k(q) = \arg \max_{e \in \mathcal{E}} \text{sim}(q, t_e)$$

While this approach captures semantic similarity, it does not enforce that retrieved entities satisfy domain-specific constraints. In practice, this leads to results that are semantically

related to the query but violate expected type constraints (e.g., returning airports when querying for route segments).

We therefore define ontology-constrained retrieval as the problem of retrieving entities that are both semantically relevant and type-consistent with a target ontology class (or set of classes) $\mathcal{C}$. Formally, we seek:

$$\text{Top}_k^{\mathcal{C}}(q) = \arg \max_{e \in \mathcal{E}, T_e \cap \mathcal{C} \neq \emptyset} \text{sim}(q, t_e)$$

The goal is to ensure that all retrieved entities satisfy the ontological constraint $\mathcal{C}$ while preserving as much semantic relevance as possible. This formulation highlights the key challenge addressed in this work: integrating symbolic ontology constraints into vector-based retrieval without sacrificing efficiency.

IV. METHOD

We consider two conceptual integration points for ontology-constrained vector retrieval, shown in Fig. 1: (i) query translation using the ontology before retrieval (Location C), and (ii) tighter integration of ontology-aware logic within the database engine itself (Location B).

In this preliminary implementation, we instantiate the **Location C** approach using pre-hoc filtering over a vector database. Rather than applying ontology constraints after retrieval, we compile ontological type information into vector database payload metadata and use these constraints to restrict the candidate set before similarity ranking.

Our implementation proceeds in four stages. First, we parse NASA’s ATM ontology and associated instance data from Turtle files using `rdflib`. Second, we extract typed ABox entities and compute their transitive superclass closure using the ontology’s `rdfs:subClassOf` hierarchy. Third, we construct an *entity card* for each indexed entity by combining its label, ontological type information, and selected attributes such as coordinates, identifiers, and selected domain-specific metadata. These entity cards are then embedded into dense vector representations using a pretrained sentence-transformer model.

Finally, the entity embeddings are indexed in Qdrant, a vector database supporting approximate nearest-neighbor search with structured payload filtering. For each entity, we store both its vector representation and associated ontology-derived payload fields, including:

- `iri`: the canonical ontology identifier,
- `types`: direct class assertions, and
- `types_closure`: direct types plus all superclasses reachable via `rdfs:subClassOf`.

At query time, the natural language query is embedded using the same sentence-transformer model. A corresponding ontological type constraint is then converted into a Qdrant payload filter over `types_closure`. Qdrant applies this filter before similarity ranking, thereby ensuring that nearest-neighbor search is performed only over ontologically valid candidates.

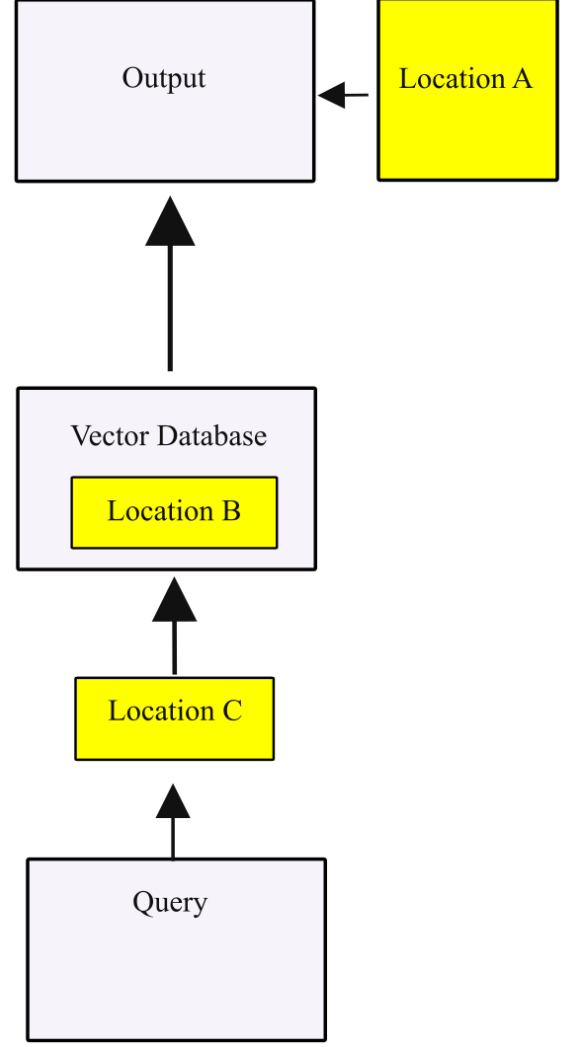


Fig. 1. Ontology-constrained vector retrieval pipeline. Prior work applies ontology constraints as a post-hoc filter (Location A). We instead use the ontology to restrict the candidate set prior to retrieval (Location C), or more tightly integrate constraints within the database (Location B). This work implements the Location C approach using payload filtering in a vector database.

This design yields a practical hybrid retrieval architecture in which symbolic ontology structure enforces semantic validity, while vector similarity provides flexible semantic ranking. Furthermore, it requires no modification to the underlying vector database, enabling straightforward integration with existing systems.

V. EXPERIMENTAL SETUP

We evaluate ontology-constrained vector retrieval using a prototype system built over NASA’s Air Traffic Management ontology (ATMONT) and associated National Airspace System data [11]. The source ontology consists of 29 Turtle files containing approximately 2.93 million RDF triples, spanning

both schema-level ontology structure (TBox) and large-scale instance data (ABox).

A. Entity Extraction and Representation

We parse the ontology using `rdflib` and extract typed ABox entities for indexing. In the current enriched prototype, we index 36,655 entities drawn from six major type families:

- `ATM#IntersectionFix`
- `ATM#LatLonFix`
- `ATM#AirspaceRouteSegment`
- `NAS#InternationalAirport`
- `data#METARreport`
- `data#TAFmeteorologicalCondition`

For each entity, we compute its direct types and its transitive superclass closure using the ontology’s `rdfs:subClassOf` graph. We then construct a textual entity description containing its label, type hierarchy, and selected domain attributes. These attributes include, when available, coordinates, airport names and codes, fix identifiers, and selected meteorological fields.

B. Embeddings and Vector Index

Entity descriptions are embedded using the pretrained sentence-transformer model `all-MiniLM-L6-v2`, which produces 384-dimensional dense vectors. These vectors are indexed in a local Qdrant instance deployed via Docker.

Each indexed point in Qdrant stores:

- a 384-dimensional embedding vector,
- an entity identifier (`iri`),
- direct ontology types (`types`), and
- transitive type closure (`types_closure`).

C. Retrieval Conditions

We evaluate two retrieval modes:

- **Unconstrained retrieval:** standard top- k nearest-neighbor search over the full index.
- **Ontology-constrained retrieval:** nearest-neighbor search restricted using a Qdrant payload filter over `types_closure`.

We use a set of ten natural language queries reflecting realistic air traffic information needs, including navigation fixes, route segments, airport entities, and weather products. Each query is paired with a target ontology class or class family representing valid result types.

All experiments use $k = 5$. We report:

- **valid@5:** proportion of top-5 results satisfying the target ontology constraint,
- **avg_score@5:** mean similarity score of the top-5 returned results, and
- **query time:** total and average latency across queries.

VI. RESULTS

Table I summarizes retrieval performance across all queries.

Ontology-constrained retrieval achieves perfect type validity across all queries, improving mean valid@5 from 0.82 to 1.00. In contrast, unconstrained retrieval frequently returns semantically similar but ontologically invalid entities.

TABLE I
RETRIEVAL PERFORMANCE COMPARISON ($k=5$).

Metric	Unconstrained	Constrained
Mean valid@5	0.82	1.00
Mean avg_score@5	0.5065	0.4882
Total query time (s)	0.1380	0.1967
Avg query time (s)	0.0138	0.0197

This behavior is most evident in route segment queries, where unconstrained retrieval returns only airport entities, while constrained retrieval returns only valid route segments.

While constrained retrieval slightly reduces the average similarity score (0.4882 vs. 0.5065), this reflects a shift from purely geometric similarity toward semantically valid results. The decrease in similarity is therefore expected, as the search space is restricted to ontologically valid candidates.

In terms of performance, ontology constraints introduce a modest latency overhead, increasing average query time from 13.8 ms to 19.7 ms. This overhead is small relative to the improvement in semantic correctness.

VII. ANALYSIS

We analyze the behavior of unconstrained and ontology-constrained retrieval to better understand the observed performance differences.

A. Failure Modes of Unconstrained Retrieval

Unconstrained vector retrieval relies solely on embedding similarity, which captures statistical and contextual relationships in the training data. As a result, it often prioritizes entities that are strongly associated with prominent query terms, even when those entities are semantically inappropriate for the task.

For example, queries containing airport identifiers such as “JFK” tend to retrieve airport entities regardless of the intended query type. This occurs because embedding models strongly associate such tokens with their most frequent semantic contexts (e.g., airports), rather than with structurally constrained concepts such as route segments or navigation fixes.

This explains the observed failure in route segment queries, where unconstrained retrieval returns only airport entities, yielding zero valid results.

B. Effect of Ontology Constraints

Ontology constraints alter the retrieval process by restricting the candidate set prior to similarity ranking. Formally, retrieval is performed over a filtered subset of entities satisfying the ontology constraint, rather than over the full entity set.

This has two important effects. First, it guarantees semantic validity by eliminating all entities that violate the specified type constraint. Second, it modifies the ranking distribution by removing high-scoring but invalid candidates, allowing lower-scoring but valid entities to surface.

This mechanism explains the empirical tradeoff observed in Table I: ontology-constrained retrieval improves valid@5

from 0.82 to 1.00, while slightly reducing average similarity scores. The decrease in similarity reflects the exclusion of semantically strong but ontologically invalid matches.

C. Qualitative Example

The query “*arrival route segment near JFK*” illustrates the difference between unconstrained and ontology-constrained retrieval.

Unconstrained retrieval (top-5):

- JFK Airport
- EFKJ Airport
- EFKT Airport
- EFKA Airport
- EFKK Airport

Ontology-constrained retrieval (top-5):

- J518 Subpath07620120908_01
- NKS174-201407150027_AR16
- NKS174-201407150027_J191
- JBU582-201407150007_J37
- JBU576-201407150034_J121

Under unconstrained retrieval, all returned entities are airports, reflecting strong semantic association with the token “JFK.” None satisfy the intended query type, yielding zero valid results. In contrast, ontology-constrained retrieval returns only valid `AirspaceRouteSegment` entities.

This example highlights the core limitation of embedding-based retrieval: semantic proximity alone does not enforce structural correctness. Ontology constraints provide a mechanism to ensure that retrieved entities satisfy domain-specific requirements.

VIII. DISCUSSION

The results demonstrate a key limitation of unconstrained vector retrieval: high semantic similarity does not guarantee semantic correctness. Embedding-based search frequently retrieves entities that are lexically or contextually related to the query but violate domain-specific constraints.

Ontology-constrained retrieval achieves perfect type validity across all queries, improving mean valid@5 from 0.82 to 1.00. In the constrained condition, ontology-derived type constraints are enforced directly through Qdrant payload filtering over `types_closure`, ensuring that all retrieved entities satisfy the target class condition before similarity ranking is performed.

However, the current system does not incorporate geographic reasoning. Queries involving spatial relationships (e.g., “near JFK” or “near New York”) rely solely on embedding similarity rather than explicit spatial filtering. As a result, retrieved entities may satisfy type constraints but lack true geographic relevance.

Additionally, the quality of constraints depends on the correctness of ontology usage. For example, restricting airport queries to the class *InternationalAirport* (meaning airports outside the United States in the ontology) can produce semantically incorrect results when querying for airports within

the United States. This highlights the importance of aligning ontological constraints with query intent.

Future work will explore integrating spatial reasoning and more expressive query translation mechanisms to improve alignment between natural language queries and ontology constraints.

IX. FUTURE WORK

Our current system demonstrates that ontology-constrained vector retrieval significantly improves type validity over unconstrained semantic search. To extend this work, we consider more expressive and formally-constrained approaches as natural extensions of this work.

A. Formal Ontology Reasoning via Description Logics

A natural next step for our work would be to leverage a Description Logics (DL) reasoner. The ATM ontology is grounded in OWL, whose semantics are defined by these description logics. Integrating a DL reasoner would enable implicit type materialization: inferring that an entity belongs to a class based on its properties, even when that membership is not explicitly asserted. This would enrich our types closure beyond what structural graph traversal captures, reducing false negatives in constrained retrieval.

B. Tableau-Based Consistency Verification

Another large goal of our project is verification of retrieval. Tableau-based reasoning could serve as this post-retrieval verification step: after candidate entities are retrieved via vector search, a reasoner would verify that each candidate is logically consistent with the query constraints and the broader ontology before returning results. This would allow the system to reject entities that satisfy type filters but violate deeper ontological axioms.

C. Consistent Sub-World Reasoning

Our dataset comes with no guarantee of being absolutely correct. As a real-world aviation knowledge graph, there could be inconsistent data, especially when concerning time-stamped observations. When global consistency in the ontology cannot be guaranteed, reasoning such “ontology repairs,” [12] that allow the system to identify the most coherent available slice of the knowledge graph, have been studied before using techniques such as axiom weakening [13]. We could incorporate these techniques to increase the reliability of retrieval in our setting.

D. Neuro-Symbolic Retrieval Architecture

These directions contribute to the bigger picture of a unified *neuro-symbolic* retrieval architecture. Vector search would serve as the candidate generator, and formal reasoning would handle consistency.

These approaches are growing in popularity, and have been explored explicitly with domain ontologies as the role of symbolic structures” [14].

X. CONCLUSION

ACKNOWLEDGMENTS

REFERENCES

- [1] J. J. Pan, J. Li, Z. Zhao, and Y. Tang, “A survey of vector database management systems,” *arXiv preprint arXiv:2310.14021*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.14021>
- [2] T. Taipalus, “Vector database management systems: Fundamental concepts, use-cases, and current challenges,” *arXiv preprint arXiv:2309.11322*, 2023. [Online]. Available: <https://arxiv.org/abs/2309.11322>
- [3] B. Smith, “Ontology,” *The Blackwell Guide to the Philosophy of Computing and Information*, pp. 153–166, 2004.
- [4] I. Niles and A. Pease, “Towards a standard upper ontology,” in *Proceedings of the International Conference on Formal Ontology in Information Systems - Volume 2001*, ser. FOIS '01. New York, NY, USA: Association for Computing Machinery, 2001, p. 2–9. [Online]. Available: <https://doi.org/10.1145/505168.505170>
- [5] T. Neumann and G. Weikum, “Rdf-3x: A risc-style engine for rdf,” in *Proceedings of the VLDB Endowment*, vol. 1, no. 1, 2008, pp. 647–659.
- [6] R. Cyganiak, D. Wood, and M. Lanthaler, “Rdf 1.1 concepts and abstract syntax,” <https://www.w3.org/TR/rdf11-concepts/>, 2014, w3C Recommendation.
- [7] W. O. W. Group, “Owl 2 web ontology language: Document overview,” <https://www.w3.org/TR/owl2-overview/>, 2012, w3C Recommendation.
- [8] L. Ma, R. Zhang, Y. Han, S. Yu, Z. Wang, Z. Ning, J. Zhang, P. Xu, P. Li, W. Ju *et al.*, “A comprehensive survey on vector database: Storage and retrieval technique, challenge,” *arXiv preprint arXiv:2310.11703*, 2023.
- [9] A. Gali, C. X. Chen, K. T. Claypool, and R. Uceda-Sosa, “From ontology to relational databases,” in *International Conference on Conceptual Modeling*. Springer, 2004, pp. 278–289.
- [10] R. W. Group, Feb 2014. [Online]. Available: <https://www.w3.org/RDF/>
- [11] NASA, “Air traffic management ontology for the national airspace system,” NASA, Tech. Rep. NASA Technical Report 20170006095, 2017. [Online]. Available: <https://ntrs.nasa.gov/citations/20170006095>
- [12] C. Meilicke, H. Stuckenschmidt, and A. Tamilin, “Repairing ontology mappings,” in *AAAI*, vol. 3, no. 2.1, 2007, p. 6.
- [13] N. Troquard, R. Confalonieri, P. Galliani, R. Penaloza, D. Porello, and O. Kutz, “Repairing ontologies via axiom weakening,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [14] A. Smirnov, A. Ponomarev, and A. Agafonov, “Ontology-based neuro-symbolic ai: effects on prediction quality and explainability,” *IEEE Access*, vol. 12, pp. 156 609–156 626, 2024.